

```

1  clc;
2  clear;
3  close all;
4
5  %% 1) Données
6  Vin = 5;           % source
7  VDD = 3.3;
8  Vref = 2.5;        % source de référence
9
10 VinH = 4.2;
11 VinL = 2.8;
12
13 R3 = 1e6;
14 R4 = 1e6;
15
16
17 %% 2) Calculs
18 x = (Vref * VDD) / ((Vref*VinL) + (VDD * VinH) - (VinH * Vref));
19 y = (VinH * VDD) / ((Vref*VinH) - (Vref * VinL) + (Vref * VDD));
20
21 z = (y / x) * (Vref / VinH);
22
23 if x >= 1
24     x = 1;
25     R1 = 0;
26     R2 = 0;
27 else
28     R1 = R3 * ((1 - z) / (x * z));
29     R2 = R1 * (x / (1 - x));
30 end
31
32 if y >= 1
33     y = 1;
34     R4 = 0;
35     R5 = 0;
36 else
37     R5 = R4 * (y / (1 - y));
38 end
39
40 %% Normalisation
41 R1_n = NormaliseComp(R1, 24);
42 R2_n = NormaliseComp(R2, 24);
43 R3_n = NormaliseComp(R3, 24);
44 R4_n = NormaliseComp(R4, 24);
45 R5_n = NormaliseComp(R5, 24);
46
47 x_n = R2_n / (R1_n + R2_n);
48
49 if (R4_n + R5_n) ~= 0
50     y_n = R5_n / (R4_n + R5_n);
51 else
52     y_n = 1;
53 end
54
55 z_n = R3_n / (R1_n * x_n + R3_n);
56
57 VinH_n = (y_n * Vref) / (x_n * z_n);
58 VinL_n = (y_n * Vref - (1 - z_n) * VDD) / (x_n * z_n);
59
60 %% Tolérances
61 R1_t = R1_n      * [1.01 1 1 1 1 1 1];

```

```

62 R2_t = R2_n      * [1 1.01 1 1 1 1 1];
63 R3_t = R3_n      * [1 1 1.01 1 1 1 1];
64 R4_t = R4_n      * [1 1 1 1.01 1 1 1];
65 R5_t = R5_n      * [1 1 1 1 1.01 1 1];
66 Vref_t = Vref * [1 1 1 1 1 1.01 1];
67 VDD_t = VDD * [1 1 1 1 1 1 1.01];
68
69 x_t = R2_t ./ (R1_t + R2_t);
70
71
72 % y dépend de R4/R5 : si branche supprimée -> y=1
73 y_t = ones(size(x_t));
74 idx = (R4_t + R5_t) ~= 0;
75 y_t(idx) = R5_t(idx) ./ (R4_t(idx) + R5_t(idx));
76
77 z_t = R3_t ./ (x_t .* R1_t + R3_t);
78
79 VinH_t = y_t .* Vref_t ./ (x_t .* z_t);
80 VinL_t = (y_t .* Vref_t - (1 - z_t) .* VDD_t) ./ (x_t .* z_t);
81
82 errH = abs(VinH_t - VinH) * 1000;
83 errL = abs(VinL_t - VinL) * 1000;
84
85 VinH_min = min(VinH_t); VinH_max = max(VinH_t);
86 VinL_min = min(VinL_t); VinL_max = max(VinL_t);
87
88 %% Résultats
89 fprintf('\n===== RESULTATS CALCUL =====\n');
90
91 fprintf('\n--- Coefficients ---\n');
92 fprintf('x = %.5f\n', x);
93 fprintf('y = %.5f\n', y);
94 fprintf('z = %.5f\n', z);
95
96 fprintf('\n--- Coefficients normalisés---\n');
97 fprintf('x_n = %.5f\n', x_n);
98 fprintf('y_n = %.5f\n', y_n);
99 fprintf('z_n = %.5f\n', z_n);
100
101 fprintf('\n--- Resistances ---\n');
102 fprintf('R1 = %.3f kOhm\n', R1/1e3);
103 fprintf('R2 = %.3f kOhm\n', R2/1e3);
104 fprintf('R3 = %.3f kOhm\n', R3/1e3);
105 fprintf('R4 = %.3f kOhm\n', R4/1e3);
106 fprintf('R5 = %.3f kOhm\n', R5/1e3);
107
108 fprintf('\n--- Resistances normalisees (E24) ---\n');
109 fprintf('R1_n = %.3f kOhm\n', R1_n/1e3);
110 fprintf('R2_n = %.3f kOhm\n', R2_n/1e3);
111 fprintf('R3_n = %.3f kOhm\n', R3_n/1e3);
112 fprintf('R4_n = %.3f kOhm\n', R4_n/1e3);
113 fprintf('R5_n = %.3f kOhm\n', R5_n/1e3);
114
115 fprintf('\n--- Vin et Vin_n ---\n');
116 fprintf('VinH = %.5f\n', VinH);
117 fprintf('VinL = %.5f\n', VinL);
118 fprintf('VinH_n = %.5f\n', VinH_n);
119 fprintf('VinL_n = %.5f\n', VinL_n);
120
121
122 names = {'R1', 'R2', 'R3', 'R4', 'R5', 'Vref', 'VDD'};

```

```
123
124 Tol_results = [VinH_t' errH' VinL_t' errL']
125
126 fprintf('\n===== \n\n');
127
128
129
130
131
```